

WEEK 04 ,05

Title

Implementation of Classes, Objects and Constructor Overloading in Java

Objective

To develop Java programs to demonstrate:

1. **Classes and Objects** using an Animal information system.
2. **Constructor Overloading** using an Employee management system.

System Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and Monitor

Software Requirements

- JDK 8 or above
 - VS Code / Eclipse / IntelliJ IDEA
 - Command Prompt / PowerShell
-

PART A: Animal Information using Classes and Objects

Algorithm / Logic Description

1. Create an `Animal` class with name, stay, food and sound attributes.
2. Create a constructor to initialize the animal details.
3. Create a `display()` method to display the details.

4. Create objects for Cow, Pig and Horse.
5. Call the `display()` method for each object.
6. Display the animal information.

Source Code

Save as **ClassObjLabPro.java**

```
// Animal Class
class Animal {
    String name;
    String stay;
    String food;
    String sound;

    // Constructor
    Animal(String name, String stay, String food, String sound) {
        this.name = name;
        this.stay = stay;
        this.food = food;
        this.sound = sound;
    }

    // Display method
    void display() {
        System.out.println("Animal : " + name);
        System.out.println("Stays : " + stay);
        System.out.println("Eats : " + food);
        System.out.println("Sound : " + sound);
        System.out.println();
    }
}

// Main Class
public class ClassObjLabPro {
    public static void main(String[] args) {

        // Creating Animal objects
        Animal cow = new Animal("Cow", "Cowshed", "Grass", "Moo");
        Animal pig = new Animal("Pig", "Pigsty", "Vegetables", "Oink");
        Animal horse = new Animal("Horse", "Stable", "Grass", "Neigh");

        // Displaying animal details
        cow.display();
```

```
        pig.display();  
        horse.display();  
    }  
}
```

Compilation and Execution

```
javac ClassObjLabPro.java  
java ClassObjLabPro
```

Output

Animal : Cow
Stays : Cowshed
Eats : Grass
Sound : Moo

Animal : Pig
Stays : Pigsty
Eats : Vegetables
Sound : Oink

Animal : Horse
Stays : Stable
Eats : Grass
Sound : Neigh

PART B: Employee Management using Constructor Overloading

Algorithm / Logic Description

1. Create an **Employee** class with employee details.
2. Create one constructor with name and ID.
3. Create another constructor with name, ID, designation, salary and promotion status.
4. Create employee objects using both constructors.
5. Call the **display()** method.
6. Display the employee details.

Source Code

Save as **Employee.java**

```
// Employee Class
public class Employee {
    String name;
    int id;
    String designation;
    double salary;
    String promotionStatus;

    // Constructor with name and id
    Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    // Overloaded Constructor
    Employee(String name, int id, String designation,
        double salary, String promotionStatus) {
        this.name = name;
        this.id = id;
        this.designation = designation;
        this.salary = salary;
        this.promotionStatus = promotionStatus;
    }

    // Display method
    void display() {
        System.out.println(name);
        System.out.println(id);
        System.out.println(designation);
        System.out.println(salary);
        System.out.println(promotionStatus);
    }

    // Main method
    public static void main(String[] args) {

        // Creating Employee objects
        Employee e1 = new Employee("Naveen", 101);
        Employee e2 = new Employee(
            "Rahul", 102, "Manager", 50000, "Yes"
        );
    }
}
```

```

        // Displaying employee details
        e1.display();
        e2.display();
    }
}

```

Compilation and Execution

```

javac Employee.java
java Employee

```

Output

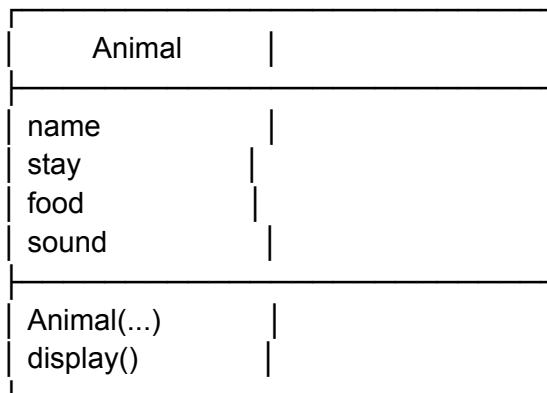
```

Naveen
101
null
0.0
null
Rahul
102
Manager
50000.0
Yes

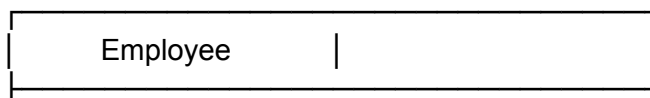
```

UML Class Diagram

Part A – Animal



Part B – Employee



name		
id		
designation		
salary		
promotionStatus		
Employee(name, id)		
Employee(name, id, ...)		
display()		
main(String[])		

```
PS C:\Users\Naveen\OneDrive\Desktop\javaprograms\Labprograms\Week 4,5> java ClassObjLabPro
Animal : Cow
Stays  : Cowshed
Eats   : Grass
Sound  : Moo

Animal : Pig
Stays  : Pigsty
Eats   : Vegetables
Sound  : Oink

Animal : Horse
Stays  : Stable
Eats   : Grass
Sound  : Neigh
```

Naveen

101

null

0.0

null

Rahul

102

Manager

50000.0

Yes